

Documento de Diseño

SICORRE / Sistema de Control de Registros para la Restitucion de

Derechos de niños, niñas y adolescentes

Versión de documento 1.0

16 de junio de 2026

[Pulido Robles Daniel Alfonso, Alison Beatriz Estevez Perez/ NULL]

Índice general

1	Organización del Diseño	2
1.1	Lineamientos de Diseño	2
1.2	Metodología o Procedimiento o Técnicas	3
1.3	Notación	5
1.4	Alcance	5
2	Diseño Arquitectónico	8
2.1	Propósito y Objetivo de la Arquitectura	8
2.2	Diagrama Arquitectónico	8
2.3	Explicación de la Arquitectura	8
2.3.1	Elementos	8
2.3.2	Relaciones entre elementos	11
2.3.3	Reglas arquitectónicas	11
2.4	Beneficios y Limitaciones de la Arquitectura	12
2.4.1	Beneficios	12
2.4.2	Limitaciones	12
3	Diseño Estático	15
3.1	Descripción y Alcance del Diseño	15
3.2	Diseño de Subsistemas	15
3.3	Diseño de Módulos	15
3.4	Diseño de Paquetes	15
3.5	Diseño de Clases	16
4	Diseño Dinámico	17
4.1	Diseño Dinámico del CU: Login	17
5	Diseño de la Persistencia de la Información	18
5.1	Modelo Relacional	18
5.2	Diccionario de Datos	18
5.3	Diseño de las Consultas	18

1. Organización del Diseño

El presente capítulo establece el marco general que rige la elaboración de este documento de diseño para el **Sistema de Control de Registros para la Restitución de Derechos de Niños, Niñas y Adolescentes (SICORRE)**. Se definen los lineamientos, la metodología, la notación y el alcance adoptados, con el fin de garantizar coherencia, trazabilidad y calidad a lo largo de todas las fases del ciclo de vida del proyecto.

1.1 Lineamientos de Diseño

El diseño de SICORRE está regido por un conjunto de principios y directrices que aseguran que el sistema responda tanto a las necesidades operativas de la Fundación de Restitución de Derechos de Niños, Niñas y Adolescentes (en adelante, *la Fundación*) como a los estándares técnicos y éticos exigidos por la naturaleza sensible de la información que gestiona.

Principios rectores

- **Centrado en el usuario.** El sistema debe ser intuitivo y accesible para el personal operativo de la Fundación, quienes en su mayoría provienen de perfiles de trabajo social y psicología, no necesariamente técnicos.
- **Confidencialidad y protección de datos.** Dado que SICORRE gestiona expedientes, diagnósticos y datos personales de menores en situación de vulnerabilidad, todo el diseño debe contemplar controles de acceso basados en roles (RBAC), cifrado en tránsito (TLS/HTTPS).
- **Separación de responsabilidades.** La arquitectura cliente-servidor (React + FastAPI + PostgreSQL) impone una separación clara entre presentación, lógica de negocio y persistencia, lo que facilita el mantenimiento y la evolución independiente de cada capa.
- **Trazabilidad y auditoría.** Toda operación de creación, modificación o consulta de un expediente debe quedar registrada con el identificador del usuario responsable, la fecha-hora y la acción realizada, garantizando un historial completo de cada caso.
- **Escalabilidad y mantenibilidad.** El diseño modular permitirá incorporar nuevas funcionalidades (p.ej., módulos de reportes estadísticos, alertas automáticas) sin impactar los módulos existentes.

- **Disponibilidad y tolerancia a fallos.** Se definirán estrategias de respaldo periódico de la base de datos PostgreSQL y se documentarán los procedimientos de recuperación ante desastres.

Estándares y normas adoptados

- **REST / OpenAPI 3.x** para la definición y documentación de los endpoints expuestos por el backend FastAPI.
- **PEP 8** como guía de estilo para el código Python del backend.
- **Airbnb Style Guide** como convención de estilo para el código JavaScript/React del frontend.
- **Conventional Commits** para el registro de cambios en el repositorio de control de versiones (Git).

Restricciones de diseño

- El sistema deberá operar en la infraestructura existente de la Fundación o en un entorno de nube de bajo costo, por lo que el diseño evitará dependencias de plataformas propietarias de alto costo o terceros, enfocándose en un entorno donde solo tenga acceso personal autorizado de la organización.
- El backend será desarrollado exclusivamente con **FastAPI** (Python ≥ 3.11) y el frontend con **React** (Node.js ≥ 20 LTS), utilizando **PostgreSQL 15** como motor de base de datos relacional.
- La comunicación entre el frontend y el backend se realizará únicamente a través de la API REST documentada; no se permitirán accesos directos a la base de datos desde el cliente, no se permitira acceso a la API por terceros.
- El sistema deberá soportar al menos 30 usuarios concurrentes en la fase inicial de despliegue.

1.2 Metodología o Procedimiento o Técnicas

Metodología de diseño

El diseño de SICORRE adopta un enfoque **orientado a componentes** alineado con el patrón arquitectónico **MVC (Modelo-Vista-Controlador)**, adaptado a la separación física de capas que impone la arquitectura elegida:

- **Modelo:** representado por los esquemas de base de datos en PostgreSQL y los modelos ORM definidos con *SQLAlchemy* en el backend.

- **Vista:** implementada como una Single Page Application (SPA) en *React*, que consume los recursos expuestos por la API REST.
- **Controlador:** materializado en los *routers* y *servicios* de FastAPI, que encapsulan la lógica de negocio y orquestan el acceso a los datos.

El proceso de diseño sigue un ciclo iterativo-incremental, en el cual cada módulo funcional (gestión de expedientes, diagnósticos, usuarios, reportes) es diseñado, prototipado, revisado y refinado antes de pasar a la fase de construcción.

Técnicas empleadas

- **Modelado de datos relacional** mediante diagramas Entidad-Relación (ER) para la definición del esquema PostgreSQL.
- **Diseño de API-first:** los contratos de la API REST son definidos en OpenAPI antes de implementar los endpoints, facilitando el desarrollo paralelo del frontend y el backend.
- **Diagramas de casos de uso UML** para capturar las interacciones entre actores (trabajador social, coordinador, administrador del sistema) y el sistema.
- **Diagramas de secuencia UML** para modelar los flujos de operación críticos, como el registro de un nuevo expediente o la actualización de un diagnóstico.
- **Wireframes y prototipos de baja fidelidad** para validar la experiencia de usuario antes de la implementación de la interfaz en React o por su parte HTML antes de desarrollo y aprobación.

Herramientas

Cuadro 1.1: Herramientas utilizadas en el diseño de SICORRE

Categoría	Herramienta	Propósito
Modelado UML	draw.io / PlantUML	Diagramas de casos de uso, secuencia, etc.
Diseño de API	Swagger UI / OpenAPI 3.x	Especificación y documentación de endpoints
Modelado de BD	dbdiagram.io / pgAdmin	Diseño del esquema relacional
Prototipado UI	Figma / HTML	Wireframes y flujos de navegación
Control de versiones	Git / GitHub	Gestión del código fuente
Gestión del proyecto	Teams / whatsapp	Seguimiento de tareas e iteraciones
Documentación	L ^A T _E X	Elaboración del presente documento

1.3 Notación

La totalidad de los diagramas incluidos en este documento de diseño se elaboran conforme al estándar **UML 2.x** (*Unified Modeling Language*, versión 2 o superior). A continuación se describe brevemente cada tipo de diagrama empleado y el propósito que cumple dentro del documento.

Cuadro 1.2: Tipos de diagramas UML utilizados en SICORRE

Tipo de diagrama	Notación principal	Uso en este documento
Casos de uso	Actor, caso de uso (elipse), relaciones	Describir interacciones entre usuarios y el sistema
Clases	Clases, atributos, métodos, asociaciones	Modelar entidades del dominio y sus relaciones
Secuencia	Líneas de vida, mensajes, activaciones	Detallar flujos de operaciones y llamadas entre capas
Componentes	Componentes, interfaces, dependencias	Representar la arquitectura lógica del sistema
Despliegue	Nodos, artefactos, comunicaciones	Describir la infraestructura física de ejecución
Entidad-Relación (ER)	Entidades, atributos, relaciones (crow's foot)	Modelar el esquema de la base de datos PostgreSQL

Adicionalmente, para la documentación de los endpoints de la API REST se utiliza la notación **OpenAPI 3.x**, que define de forma estandarizada los recursos, métodos HTTP, parámetros, cuerpos de solicitud y códigos de respuesta.

1.4 Alcance

Descripción general

El presente documento de diseño cubre el análisis y especificación arquitectónica del **Sistema de Control de Registros para la Restitución de Derechos de Niños, Niñas y Adolescentes (SICORRE)**, desarrollado para la Fundación de Restitución de Derechos de Niños, Niñas y Adolescentes en estado de orfandad debido al feminicidio en México.

SICORRE tiene como propósito digitalizar y estructurar los procesos actualmente gestionados en papel, tales como la creación y seguimiento de expedientes, el registro de diagnósticos, la gestión de fechas y citas, y el control de archivos relacionados con cada caso de NNA.

Módulos y funcionalidades dentro del alcance

Los siguientes subsistemas y módulos son cubiertos por este documento de diseño:

1. **Gestión de usuarios y roles:** administración de cuentas de acceso, asignación de roles (administrador, coordinador, trabajador social) y control de permisos.
2. **Gestión de equipos multidisciplinarios** administración de equipos compuestos por diferentes usuarios y roles previamente registrados para dar seguimiento a los diagnosticos y planes para la resitución de derechos.
3. **Gestión y registro de tutores** : administración y nuevos registros para tutores con sus respectivos datos de contacto y asociación con los nnas.
4. **Gestión y registro de nnas** : Gestión y registro principal de niños niñas y adolescentes en el sistema.
5. **Modulo de Restitución de derechos** : Modulo utilizado para consultar programas y actores en materia de derechos que sirvan de ayuda para la restitución de los mismos para los nnas.
6. **Consulta de catalogos** : consultas de catalogos establecidos previamente por la organización como roles existentes, enfermedades, discapacidades, idiomas, domicilios (SEPOMEX).
7. **API REST:** especificación completa de los servicios que expone el backend FastAPI para consumo del frontend React.
8. **Esquema de base de datos:** diseño del modelo relacional en PostgreSQL que soporta todos los módulos anteriores.

Elementos fuera del alcance

Los siguientes aspectos **no** son cubiertos por el presente documento de diseño:

- Integración con sistemas externos de gobierno (SIPINNA, DIF, Fiscalía) en la fase inicial; podrá contemplarse en versiones futuras.
- Módulo de comunicación directa con familias o tutores (mensajería, portal ciudadano).
- Aplicación móvil nativa; la solución es exclusivamente web responsiva.
- Módulo de firma electrónica avanzada o firma digital con valor legal.
- Infraestructura de despliegue en producción (configuración de servidores, balanceo de carga, DNS).
- Migración de expedientes físicos históricos en papel; este proceso se realizará de forma manual y operativa fuera del alcance del sistema o por asistencia de modelos de IA propuestos hechos.

La Tabla [1.3](#) presenta un resumen del alcance por capa de la arquitectura.

Cuadro 1.3: Resumen del alcance por capa arquitectónica

Capa	Tecnología	Cubierta en este doc.
Presentación (UI)	React (Node.js)	✓
Lógica de negocio	FastAPI (Python)	✓
Persistencia	PostgreSQL 15	✓
Infraestructura	Servidor / Nube	×
Integraciones externas	APIs gubernamentales	×

2. Diseño Arquitectónico

El diseño arquitectónico de SICORRE establece la estructura de alto nivel del sistema, definiendo las capas que lo componen, las tecnologías asignadas a cada una y las reglas de comunicación entre ellas. Este capítulo describe el propósito de la arquitectura elegida, presenta su representación visual, explica cada uno de sus elementos y concluye con un análisis de los beneficios y limitaciones que conlleva.

2.1 Propósito y Objetivo de la Arquitectura

La arquitectura de SICORRE tiene como propósito central proveer una plataforma **segura, modular y mantenible** que permita a la Fundación digitalizar y controlar los procesos de gestión de expedientes, diagnósticos y seguimiento de casos de Niños, Niñas y Adolescentes (NNA), sustituyendo el flujo de trabajo basado en papel por uno digital, auditable y con acceso controlado por roles.

Los objetivos arquitectónicos se expresan en términos de las siguientes propiedades de calidad del sistema, tomando como referencia el modelo **ISO/IEC 25010**:

2.2 Diagrama Arquitectónico

La arquitectura de SICORRE sigue el patrón **Cliente-Servidor de tres capas** (*3-Tier Architecture*), con una separación física clara entre la capa de presentación, la capa de lógica de negocio y la capa de persistencia de datos. La Figura ?? ilustra esta estructura.

2.3 Explicación de la Arquitectura

2.3.1 Elementos

La arquitectura de SICORRE está compuesta por los siguientes elementos principales:

Capa de Presentación — React SPA

- **Tecnología:** React (biblioteca de UI de JavaScript), ejecutada sobre Node.js ≥ 20 LTS para el entorno de desarrollo y empaquetado.

Cuadro 2.1: Propiedades de calidad objetivo de la arquitectura SICORRE

Propiedad	Objetivo arquitectónico
Seguridad	Garantizar la confidencialidad e integridad de los datos de NNA mediante autenticación JWT, control de acceso basado en roles (RBAC) y comunicación cifrada TLS.
Mantenibilidad	Facilitar la evolución independiente de cada capa (presentación, lógica de negocio, persistencia) mediante bajo acoplamiento e interfaces bien definidas (API REST/OpenAPI).
Disponibilidad	Asegurar la continuidad operativa del sistema con estrategias de respaldo periódico de la base de datos y procedimientos documentados de recuperación.
Usabilidad	Proveer una interfaz web responsiva e intuitiva que no requiera formación técnica especializada por parte del personal de la Fundación.
Escalabilidad	Permitir la incorporación de nuevos módulos y el aumento de la carga de usuarios sin rediseñar la estructura base del sistema.
Trazabilidad	Registrar de forma automática cada operación sobre un expediente, garantizando un historial completo de auditoría.
Portabilidad	Asegurar que el sistema pueda desplegarse tanto en infraestructura local como en entornos de nube, sin dependencias de plataformas propietarias.

- **Responsabilidad:** Renderizar la interfaz de usuario en el navegador del cliente y gestionar la navegación entre vistas sin recarga completa de página (*Single Page Application*).
- **Componentes clave:**
 - *React Router*: manejo de rutas del lado del cliente (`/expedientes`, `/diagnosticos`, `/usuarios`, etc.).
 - *Axios / Fetch API*: cliente HTTP encargado de consumir los endpoints de la API REST del backend.
 - *Context API / Redux*: gestión del estado global de la aplicación (sesión activa, permisos del usuario, datos en caché).
 - *Formularios controlados*: captura y validación de datos antes de su envío al backend.

- **Patrón interno:** Arquitectura basada en componentes reutilizables, organizados en páginas (*pages*), componentes de UI (*components*) y servicios de acceso a la API (*services*).

Capa de Lógica de Negocio — FastAPI

- **Tecnología:** FastAPI (framework web Python asíncrono), Python ≥ 3.11 .
- **Responsabilidad:** Exponer la API REST, aplicar las reglas de negocio, gestionar la autenticación y autorización, y orquestar el acceso a la base de datos.
- **Componentes clave:**
 - *Routers:* agrupan los endpoints por dominio funcional (*/expedientes*, */diagnosticos*, */usuarios*, */reportes*).
 - *Servicios (Services):* encapsulan la lógica de negocio e interactúan con la capa de persistencia.
 - *Esquemas Pydantic:* validan y serializan los datos de entrada y salida de cada endpoint.
 - *Dependencias (Dependencies):* implementan la inyección de dependencias para la verificación de tokens JWT y la obtención de sesiones de base de datos.
 - *SQLAlchemy ORM:* abstrae el acceso a PostgreSQL mediante modelos de Python, facilitando las operaciones CRUD y las consultas complejas.
 - *Módulo de autenticación:* gestiona el ciclo de vida de los tokens JWT (*access token + refresh token*) y la validación de roles de usuario.
- **Patrón interno:** Arquitectura en capas *Router* $\rightarrow$ *Service* $\rightarrow$ *Repository* $\rightarrow$ *Model*, que separa la exposición de la API, la lógica de negocio, el acceso a datos y los modelos de dominio.

Capa de Persistencia — PostgreSQL

- **Tecnología:** PostgreSQL 15, motor de base de datos relacional de código abierto.
- **Responsabilidad:** Almacenar de forma estructurada y persistente todos los datos del sistema: expedientes, diagnósticos, usuarios, roles, documentos adjuntos (meta-datos) y registros de auditoría.
- **Componentes clave:**
 - *Esquema relacional normalizado:* tablas con relaciones de clave foránea que garantizan integridad referencial.
 - *Triggers de auditoría:* registran automáticamente las modificaciones a los expedientes en una tabla de historial.

- *Roles de base de datos*: el backend accede con un usuario de BD de mínimos privilegios; no se permiten conexiones directas desde el cliente.
- *RespalDOS*: mediante `pg_dump` programado (cron) con almacenamiento en repositorio seguro (local o S3-compatible).

Capa transversal — Seguridad

Aunque no constituye una capa física independiente, la seguridad es un elemento transversal que atraviesa todas las capas:

- **TLS/HTTPS**: cifra toda la comunicación entre el navegador y el servidor.
- **JWT (JSON Web Tokens)**: provee autenticación sin estado entre el frontend y el backend.
- **RBAC**: cada endpoint verifica el rol del usuario (administrador, coordinador, trabajador social) antes de ejecutar la operación.
- **CORS**: el backend configura las políticas de origen cruzado para permitir únicamente solicitudes del dominio autorizado del frontend.

2.3.2 Relaciones entre elementos

1. **Usuario → Capa de Presentación**: el usuario interactúa con la SPA de React a través del navegador web mediante HTTPS. No existe instalación de software en el equipo del usuario.
2. **Capa de Presentación ↔ Capa de Lógica de Negocio**: React se comunica con FastAPI exclusivamente a través de la API REST documentada en OpenAPI 3.x. Los mensajes viajan en formato JSON sobre HTTPS. Esta relación es bidireccional: el frontend envía solicitudes y el backend responde con datos o confirmaciones.
3. **Capa de Lógica de Negocio ↔ Capa de Persistencia**: FastAPI accede a PostgreSQL a través de SQLAlchemy ORM mediante el protocolo TCP/IP interno del servidor. Ningún componente externo al backend puede escribir directamente en la base de datos.
4. **Módulo de autenticación → todas las capas**: el token JWT generado en el backend es almacenado en el frontend (memoria o *httpOnly cookie*) y enviado en cada solicitud HTTP; el backend lo valida antes de procesar cualquier operación.

2.3.3 Reglas arquitectónicas

Las siguientes reglas son de cumplimiento obligatorio y rigen las interacciones entre capas durante todo el ciclo de vida del sistema:

1. **Sentido único de dependencia:** la capa de presentación depende de la capa de lógica de negocio; la capa de lógica de negocio depende de la capa de persistencia. Nunca en sentido inverso.
2. **Acceso exclusivo por API:** el frontend solo puede comunicarse con la base de datos a través de los endpoints del backend. Está prohibido exponer cadenas de conexión o credenciales de PostgreSQL en el cliente.
3. **Validación en dos niveles:** los datos de entrada son validados en el frontend (formularios React) y nuevamente en el backend (esquemas Pydantic) antes de persistirse. La validación del backend es la autoritativa.
4. **Autenticación obligatoria:** todos los endpoints de la API, excepto `POST /auth/login`, requieren un token JWT válido. El rol del usuario determina qué operaciones puede ejecutar.
5. **Sin lógica de negocio en el frontend:** las reglas de negocio (cálculo de estados, validaciones de flujo, permisos) residen exclusivamente en el backend. El frontend solo presenta datos y captura entradas.
6. **Versionado de la API:** todos los endpoints se agrupan bajo el prefijo `/api/v1/` para facilitar la evolución de la API sin romper clientes existentes.
7. **Registro de auditoría:** toda escritura sobre las tablas de expedientes y diagnósticos debe registrarse en la tabla de auditoría con usuario, timestamp y tipo de operación.

2.4 Beneficios y Limitaciones de la Arquitectura

2.4.1 Beneficios

Los siguientes beneficios se derivan directamente de las propiedades de calidad que la arquitectura de tres capas con FastAPI, React y PostgreSQL aporta al sistema:

2.4.2 Limitaciones

Toda decisión arquitectónica implica compromisos (*trade-offs*). A continuación se documentan las limitaciones identificadas y las estrategias de mitigación propuestas:

Cuadro 2.2: Beneficios de la arquitectura en términos de propiedades de calidad

Propiedad	Beneficio
Mantenibilidad	La separación estricta de capas permite modificar o reemplazar el frontend (React) sin alterar el backend, y viceversa, reduciendo el riesgo de regresiones.
Escalabilidad	FastAPI soporta ejecución asíncrona (<code>async/await</code>), lo que permite atender múltiples solicitudes concurrentes con un consumo de recursos reducido.
Seguridad	La arquitectura centraliza los controles de seguridad en el backend (JWT, RBAC, CORS), minimizando la superficie de ataque accesible desde el cliente.
Productividad	FastAPI genera automáticamente documentación interactiva (Swagger UI / ReDoc) a partir de los esquemas Pydantic, acelerando el desarrollo y las pruebas de la API.
Confiabilidad	PostgreSQL ofrece transacciones ACID, garantizando la integridad de los datos de los expedientes incluso ante fallos parciales del sistema.
Portabilidad	El uso de tecnologías de código abierto (Python, React, PostgreSQL) facilita el despliegue en múltiples entornos: servidores locales, VPS o nube pública.
Trazabilidad	La capa de auditoría en PostgreSQL registra automáticamente cada cambio en los expedientes, cumpliendo con requisitos de transparencia y rendición de cuentas.
Usabilidad	React permite construir interfaces responsivas e interactivas con experiencia de usuario fluida, sin recargas completas de página.

Cuadro 2.3: Limitaciones de la arquitectura y estrategias de mitigación

Limitación	Descripción	Mitigación
Punto único de falla	Si el servidor donde reside el backend cae, toda la aplicación deja de operar. La arquitectura de un solo servidor no ofrece redundancia nativa.	Implementar respaldos automáticos y documentar un plan de recuperación ante desastres (RTO/RPO definidos).
Latencia de red	Al ser una SPA, la carga inicial puede ser lenta en conexiones de baja velocidad, ya que el navegador descarga todo el bundle de React antes de renderizar.	Aplicar <i>code splitting</i> y carga diferida (<code>React.lazy</code>) para reducir el tamaño del bundle inicial.
Complejidad operativa	Gestionar tres capas tecnológicas (React, FastAPI, PostgreSQL) requiere competencias en múltiples lenguajes y herramientas (Python, JavaScript, SQL).	Documentar los procedimientos de despliegue y usar contenedores Docker para simplificar la configuración del entorno.
Sin modo fuera de línea	Al depender de la conexión al servidor, el sistema no puede operar sin acceso a la red. Esto puede afectar al personal en trabajo de campo.	Definir como trabajo futuro un módulo de sincronización offline (PWA o app móvil).
Escalabilidad vertical	PostgreSQL en un solo nodo puede convertirse en cuello de botella si el volumen de datos o de consultas crece significativamente.	Planificar particionamiento de tablas y, a largo plazo, migración a una solución con réplicas de lectura.
Gestión de sesiones	El uso de JWT sin estado dificulta la revocación inmediata de sesiones ante compromisos de seguridad.	Implementar una lista negra de tokens en caché (Redis o tabla en BD) con TTL controlado.

3. Diseño Estático

El diseño estático de SICORRE describe la estructura interna del código que da soporte a la arquitectura definida para el sistema, mostrando cómo se organizan los subsistemas, los módulos, los paquetes y las clases que implementan la funcionalidad descrita en la especificación de requisitos. Este capítulo presenta dicha organización mediante diagramas UML de clases y de paquetes, y detalla el propósito y las dependencias de cada elemento, de manera que sirva como referencia para la implementación y el mantenimiento del sistema a lo largo de las distintas iteraciones de desarrollo.

3.1 Descripción y Alcance del Diseño

En esta sección se detalla la estructura estática correspondiente a los módulos funcionales de SICORRE planificados para las iteraciones 1, 2 y 3, así como aquellos casos de uso que han quedado programados para iteraciones posteriores. El nivel de abstracción utilizado parte de los subsistemas que agrupan la funcionalidad del sistema, desciende a los módulos que los componen, continúa con los paquetes que organizan el código fuente de cada módulo y concluye con las clases que implementan la lógica de negocio, sus atributos, métodos y relaciones. De esta manera se ofrece una visión que va de lo general a lo particular, permitiendo ubicar cualquier elemento del código dentro de la estructura global del sistema.

3.2 Diseño de Subsistemas

La funcionalidad de SICORRE se organiza en nueve subsistemas, cada uno responsable de un conjunto cohesionado de casos de uso y expuesto al resto del sistema mediante interfaces bien definidas que evitan el acceso directo a los datos internos de otros subsistemas.

- **Control de Acceso:** responsable de la autenticación de los usuarios del sistema, así como del cierre de sesión y la recuperación de contraseña. Expone una interfaz de autenticación que es consumida por el resto de los subsistemas para validar la identidad y los privilegios del usuario en cada operación.
- **Gestión de Usuarios:** encargado del ciclo de vida de las cuentas de usuario, incluyendo su registro, consulta, modificación, revocación de acceso y eliminación. Provee la información de usuarios y roles que requiere el subsistema de Control de Acceso.

- **Equipos Multidisciplinarios:** administra el registro, consulta y modificación de los equipos multidisciplinarios, así como la asignación de niñas, niños y adolescentes (NNA) a dichos equipos, sirviendo de enlace entre el subsistema de Gestión de Usuarios y el subsistema del NNA.
- **NNA:** concentra el registro, consulta y modificación de los datos de las niñas, niños y adolescentes, el registro y modificación de su hecho victimal, y el registro y modificación de los datos de su tutor. Es consumido por los subsistemas de Equipos Multidisciplinarios y de Expedientes.
- **Expedientes:** gestiona la apertura y consulta de expedientes, así como el registro y modificación de las valoraciones multidisciplinarias y de los derechos vulnerados asociados a cada NNA, integrando la información proveniente de los subsistemas del NNA y de Equipos Multidisciplinarios.
- **Actores en Materia de Derechos:** permite el registro, consulta, modificación y eliminación de los actores en materia de derechos, el registro de los servicios que ofrecen y su consulta por municipio y tipo de servicio, exponiendo esta información al subsistema de Restitución de Derechos.
- **Catálogos:** provee información de referencia compartida por el resto del sistema, como los catálogos de domicilios (SEPOMEX) e idiomas (INALI), así como el registro y modificación de discapacidades y enfermedades del NNA o de su tutor.
- **Calendario:** administra la consulta del calendario de expedientes y el registro y modificación de eventos asociados, sirviendo de apoyo transversal a los subsistemas de Expedientes y de Equipos Multidisciplinarios.
- **Restitución de Derechos – Directorio de Actores:** concentra el registro, edición, consulta, búsqueda y filtrado de actores, sus contactos, ubicaciones, programas y enlaces de representantes, además de la consulta y búsqueda del catálogo de programas y la posibilidad de deshabilitar y reactivar tanto actores como programas.

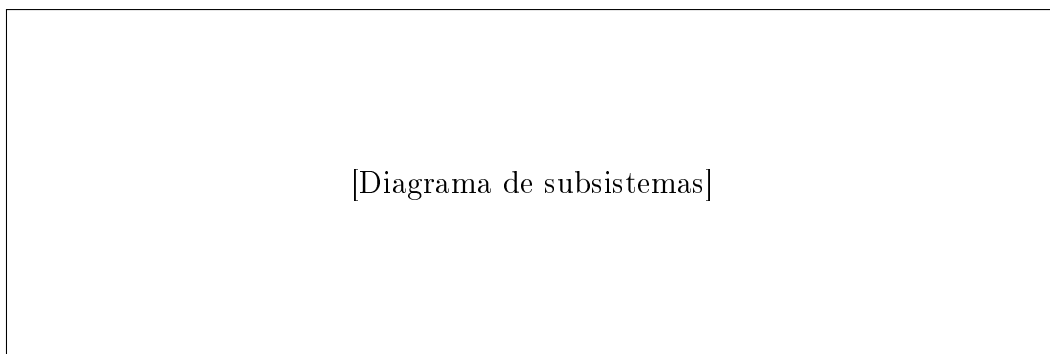


Figura 3.1: Diagrama de subsistemas.

3.3 Diseño de Módulos

Cada subsistema descrito previamente se implementa mediante uno o varios módulos de software, cuya función y casos de uso asociados se resumen a continuación junto con el estado de planificación correspondiente.

Módulo de Control de Acceso

- CU-01 Iniciar sesión – Iteración 1
- CU-02 Cerrar sesión – Iteración 1
- CU-03 Recuperar contraseña – Siguiente iteración

Depende del Módulo de Gestión de Usuarios para validar las credenciales y los privilegios de cada cuenta.

Módulo de Gestión de Usuarios

- CU-04 Registrar usuario – Iteración 1
- CU-05 Consultar usuario – Iteración 1
- CU-06 Modificar usuario – Iteración 1
- CU-07 Revocar acceso a usuario – Iteración 1
- CU-08 Eliminar usuario – Iteración 1

No depende de otros módulos para su operación básica; es consumido por el Módulo de Control de Acceso.

Módulo de Equipos Multidisciplinarios

- CU-09 Registrar equipo multidisciplinario – Iteración 2
- CU-10 Consultar equipo multidisciplinario – Iteración 2
- CU-11 Modificar equipo multidisciplinario – Iteración 2
- CU-12 Asignar NNA a equipo multidisciplinario – Iteración 2

Depende del Módulo de Gestión de Usuarios y del Módulo del NNA.

Módulo del NNA

- CU-13 Registrar NNA – Iteración 2
- CU-14 Consultar NNA – Iteración 2
- CU-15 Modificar datos del NNA – Iteración 2
- CU-16 Registrar hecho victimal – Siguiente iteración
- CU-17 Modificar hecho victimal – Siguiente iteración
- CU-18 Registrar tutor del NNA – Iteración 2
- CU-19 Modificar datos del tutor – Iteración 2

Depende del Módulo de Catálogos para la captura de domicilios, idiomas, discapacidades y enfermedades.

Módulo de Expedientes

- CU-20 Abrir expediente – Siguiente iteración
- CU-21 Consultar expediente – Siguiente iteración
- CU-22 Registrar valoración multidisciplinaria – Siguiente iteración
- CU-23 Modificar valoración multidisciplinaria – Siguiente iteración
- CU-24 Registrar derecho vulnerado – Siguiente iteración
- CU-25 Modificar derecho vulnerado – Siguiente iteración

Depende del Módulo del NNA y del Módulo de Equipos Multidisciplinarios.

Módulo de Actores en Materia de Derechos

- CU-26 Registrar actor en materia de derechos – Siguiente iteración
- CU-27 Consultar actor en materia de derechos – Siguiente iteración
- CU-28 Modificar actor en materia de derechos – Siguiente iteración
- CU-29 Eliminar actor en materia de derechos – Siguiente iteración
- CU-30 Registrar servicio de actor – Siguiente iteración
- CU-31 Consultar actores por municipio y tipo de servicio – Siguiente iteración

Depende del Módulo de Catálogos para la captura de domicilios.

Módulo de Catálogos

- CU-32 Consultar catálogo de domicilios (SEPOMEX) – Iteración 2
- CU-33 Consultar catálogo de idiomas (INALI) – Iteración 2
- CU-34 Registrar discapacidad de NNA o tutor – Iteración 2
- CU-35 Modificar discapacidad de NNA o tutor – Siguiente iteración
- CU-36 Registrar enfermedad de NNA o tutor – Iteración 2
- CU-37 Modificar enfermedad de NNA o tutor – Siguiente iteración

No depende de otros módulos; es consumido de manera transversal por el resto del sistema.

Módulo de Calendario

- CU-38 Consultar calendario de expedientes – Siguiente iteración
- CU-39 Registrar evento en calendario – Siguiente iteración
- CU-40 Modificar evento en calendario – Siguiente iteración

Depende del Módulo de Expedientes para asociar eventos a un expediente determinado.

Módulo de Restitución de Derechos – Directorio de Actores

- CU-41 Registrar actor – Iteración 3
- CU-42 Registrar contactos del actor – Iteración 3
- CU-43 Registrar ubicación del actor – Iteración 3
- CU-44 Registrar programas del actor – Iteración 3
- CU-45 Registrar enlaces representantes – Iteración 3
- CU-46 Buscar actores – Iteración 3
- CU-47 Filtrar actores por tipo – Iteración 3
- CU-48 Consultar actor – Iteración 3
- CU-49 Editar actor – Iteración 3
- CU-50 Editar contactos del actor – Iteración 3
- CU-51 Editar ubicación del actor – Iteración 3
- CU-52 Editar programas del actor – Iteración 3

- CU-53 Editar enlaces representantes – Iteración 3
- CU-54 Deshabilitar actor – Iteración 3
- CU-55 Reactivar actor – Siguiente iteración
- CU-56 Consultar catálogo de programas – Iteración 3
- CU-57 Buscar programas – Siguiente iteración
- CU-58 Deshabilitar programa – Siguiente iteración
- CU-59 Reactivar programa – Siguiente iteración

Depende del Módulo de Catálogos para la captura de domicilios y de los catálogos de programas registrados internamente.

3.4 Diseño de Paquetes

El código fuente de SICORRE se organiza en paquetes que reflejan la separación por subsistemas descrita anteriormente, agrupando en cada paquete las clases de modelo, de lógica de negocio y de acceso a datos correspondientes a un mismo módulo. Esta organización busca minimizar el acoplamiento entre módulos, de forma que las dependencias señaladas en la sección anterior se reflejen como relaciones explícitas entre paquetes, facilitando la sustitución o evolución independiente de cada módulo conforme avancen las iteraciones del proyecto.

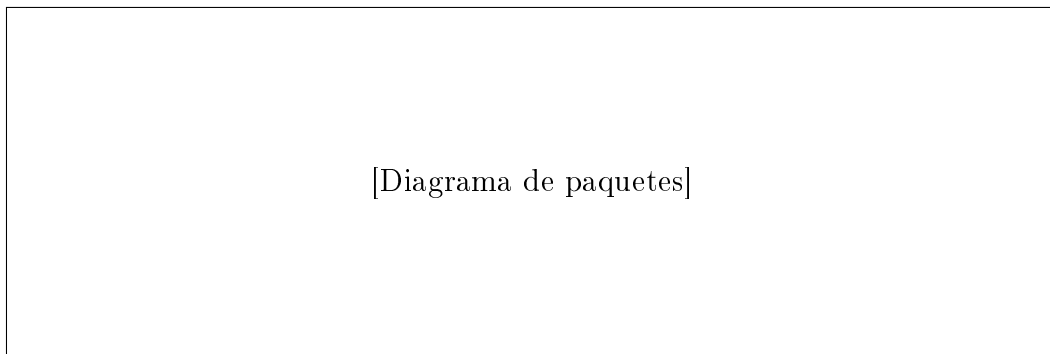


Figura 3.2: Diagrama de paquetes.

3.5 Diseño de Clases

Dentro de cada paquete se definen las clases que implementan los casos de uso correspondientes, especificando sus atributos, métodos y las relaciones de herencia, composición y dependencia que mantienen entre sí y con las clases de otros módulos. El diagrama de clases principal integra las entidades centrales del dominio, como el usuario, el equipo

multidisciplinario, el NNA, el expediente y el actor en materia de derechos, mostrando cómo se relacionan entre ellas para dar soporte a los casos de uso descritos en las secciones previas.

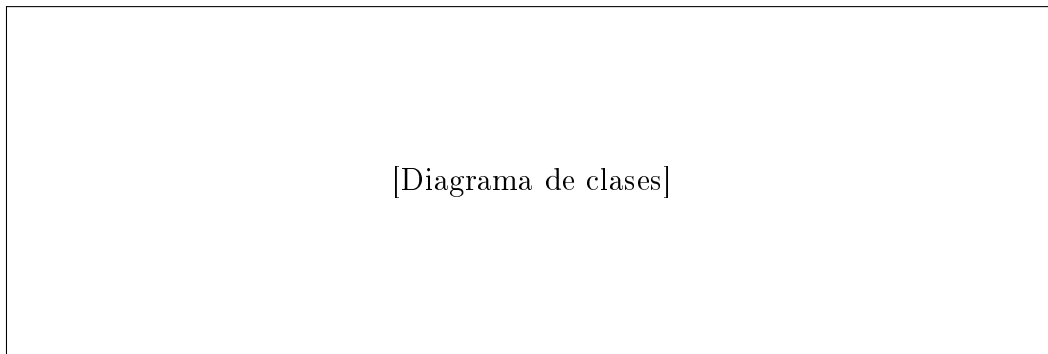


Figura 3.3: Diagrama de clases principal.

4. Diseño Dinámico

Descripción del comportamiento en tiempo de ejecución del sistema mediante diagramas de secuencia, actividad o estados.

4.1 Diseño Dinámico del CU: Login

Descripción del Flujo

El actor *Usuario* ingresa sus credenciales; el sistema las valida contra la base de datos y, según el resultado, otorga o deniega el acceso.

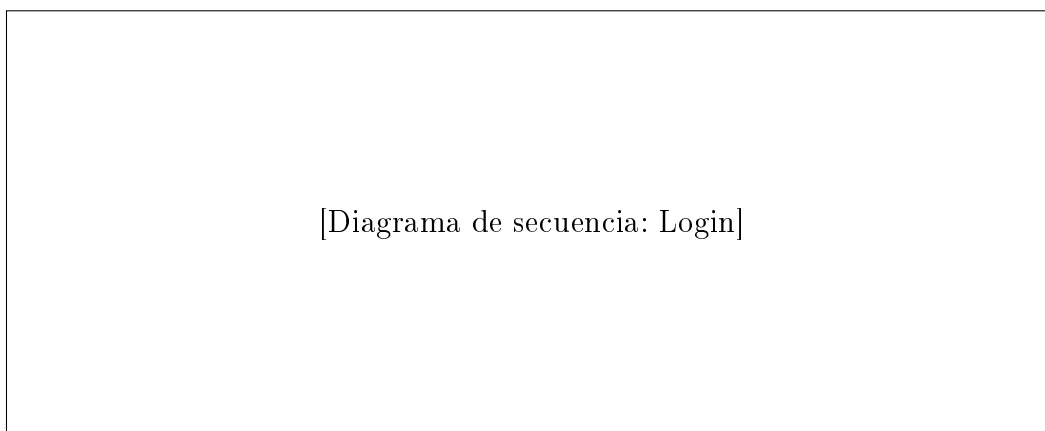


Figura 4.1: Diagrama de secuencia — CU Login.

Flujo Alternativo / Excepciones

- Credenciales incorrectas: el sistema muestra mensaje de error y permite reintentar (máximo 3 intentos).
- Cuenta bloqueada: se redirige al flujo de recuperación de contraseña.

5. Diseño de la Persistencia de la Información

5.1 Modelo Relacional

Presentar el modelo entidad-relación o el esquema relacional del sistema.

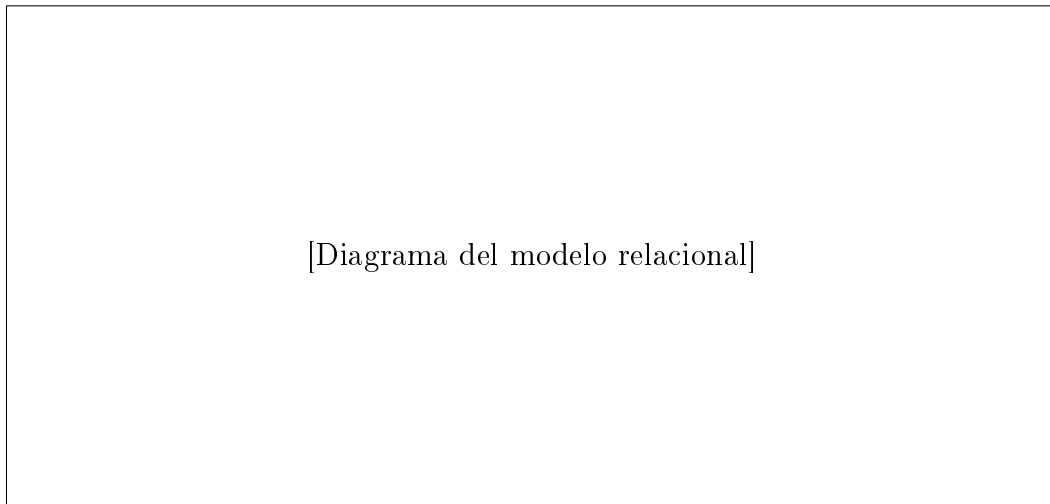


Figura 5.1: Modelo relacional de la base de datos.

5.2 Diccionario de Datos

5.3 Diseño de las Consultas

Diccionario de Consultas

Cuadro 5.1: Diccionario de datos — Tabla **usuarios**

Campo	Tipo	Longitud	Nulo	Descripción
id_usuario	INT	11	NO	Identificador único del usuario (PK).
nombre	VARCHAR	100	NO	Nombre completo del usuario.
email	VARCHAR	150	NO	Correo electrónico (único).
contrasena	VARCHAR	255	NO	Hash de la contraseña.
fecha_alta	DATETIME	–	NO	Fecha y hora de registro.
activo	TINYINT	1	NO	1 = activo, 0 = inactivo.

Cuadro 5.2: Diccionario de consultas

ID	Descripción	Sentencia SQL
Q-001	Autenticar usuario por email y contraseña	SELECT * FROM usuarios WHERE email=? AND contrasena=? AND activo=1
Q-002	Obtener perfil del usuario por ID	SELECT id_usuario, nombre, email FROM usuarios WHERE id_usuario=?
Q-003	[Descripción de la consulta]	[sentencia SQL]